

Project Defence Master Guide

Forensic scope and evidence rule. This guide was produced by read-only inspection of the repository as it existed on 8 September 2026. Application source, prompts, configuration, and existing data were not changed.

“Implemented” means observable in current code; “runtime observed” means inspected in the current `data/` or `.env`; “recommendation” is deliberately not an implementation claim. Paths are project-relative unless an absolute runtime path is stated.

1. Executive Summary

One-line definition: a Streamlit product-support prototype that turns uploaded PDF, TXT, and CSV content into local Sentence Transformer vectors in a persistent FAISS index, then uses a selected LLM only after retrieval and relevance checks to produce a cited answer.

It is a credible modular RAG prototype, not yet a production product-support platform. Its strongest implementation evidence is page/row metadata preservation, a persisted vector-to-text mapping, deterministic citations, session-only memory, explicit LangGraph control flow, and refusal when evidence is absent or weak. Its current knowledge base is **not product documentation**: it contains four sample/academic PDFs—44, 801, 120, and 2 pages respectively—and 9,351 vectors. Never claim that it has been validated on real product manuals.

Actual active runtime configuration: `LLM_PROVIDER=ollama`, `OLLAMA_MODEL=qwen3:8b`, `OLLAMA_BASE_URL=http://localhost:11434`; embeddings use `all-MiniLM-L6-v2`; chunks use 1,000 characters with 200-character overlap; FAISS is `IndexFlatIP`, 384 dimensions, with 9,351 vectors. Gemini exists as an optional provider path but is not the active provider.

2. Project Business Problem

Support users need answers grounded in approved documents rather than a model's general knowledge. The prototype addresses the small-scale version of that problem: an administrator uploads documents, a user asks a question, and the system retrieves text with source location before generating an answer. The business value is faster self-service and auditable references; the current code does **not** implement identity, tenant isolation, product/version filtering, ticket integration, or access control.

3. Product Definition, Users, and Real-World Scenario

Target roles in the present UI are a knowledge administrator (uploads/clears) and a support user (asks questions). A realistic future scenario is: product manuals, installation guides, known-issues notes, and release notes for Product A 3.2 are indexed. A user asks, “Why does E104 occur after configuration X?” Retrieval should select Product A 3.2 installation and known-issues chunks, aggregate the evidence, and return a cited answer. The current prototype can retrieve across files and selectively summarize/synthesize them, but it has no product, version, error-code schema, document permission, or conflict-resolution policy. Those are recommendations, not present features.

4. Repository Map and Entry Points

Area	Actual contents / role	Runtime status
app.py	Streamlit entry: loads settings, bootstraps, renders UI.	Active entry point: <code>streamlit run app.py</code> .
src/product_support_agent/config.py	.env loading; typed settings; all paths and thresholds.	Active.
core/bootstrap.py	Creates runtime directories, configures logging, creates empty upload manifest if needed.	Active before UI.
ui/	Streamlit layout, upload/chat screens, sidebar diagnostics, presentation state/theme.	Active.
services/	Ingestion, loaders, chunking, embeddings, vector store, retrieval, RAG, memory, prompts, lifecycle.	Active through <code>sections.py</code> .
services/llm/	Provider contract, Ollama provider, Gemini adapter/factory.	Active factory path.
models/schemas.py	Dataclass contracts for uploads, extracted units, chunks, retrieval, citations, responses.	Active.
prompts/	System, contextualization, per-document summary, multi-document synthesis prompt assets.	Active when the associated path runs.
data/uploads/	Four persisted raw PDFs at inspection time.	Active persistent storage.
data/faiss/index.faiss	14,363,181-byte FAISS index, IndexFlatIP, 9,351 vectors, d=384.	Active persistent storage.
data/faiss/index.pkl	3,424,841-byte pickled state containing chunk text, metadata, and parallel vector mapping.	Active persistent storage.
data/metadata/uploaded_files.json	JSON upload manifest; four records at inspection time.	Active persistent storage.
logs/product_support_agent.log	Rotating application log; present at inspection time.	Active once app runs.
tests/	62 static pytest tests across configuration, ingestion, retrieval, memory, RAG and Gemini.	Test-only.
README.md, *_SRS/HLD/LLD/UID.md, PDFs/DOCX	Project documentation.	Not executed.

Area	Actual contents / role	Runtime status
generate_hcl_docx.js	Documentation-generation utility; no Python call site was found.	Likely manual/legacy utility.

There are 56 Python files including tests and packages. There is no database migration, web API server, background worker, scheduled re-indexer, cache service, or separate document database in the repository.

Actual import/call chain

app.py:main → get_settings() → bootstrap_application() → ui.layout.render_app() → either ui.sections.render_upload_section() or render_rag_section(). The upload screen constructs KnowledgeBaseIngestionPipeline; the chat screen constructs RAGService plus a Streamlit-session-backed ConversationMemoryService.

GeminiService is directly tested and exposed from services/__init__.py, but UI/RAG creation uses services.llm.build_llm_service. With the observed .env, that factory returns ollamaService; Gemini is an optional adapter path, not the active one. ChatTurn, DocumentMetadata after manifest serialization, and the default prompt strings are support contracts/fallbacks rather than independently invoked runtime workflows.

5. End-to-End Architecture



LangGraph is an explicit deterministic orchestration graph, not an autonomous tool-using agent. The graph nodes are load_memory, contextualize_query, retrieve_chunks, no_results_fallback, filter_relevance, low_relevance_fallback, determine_reasoning_strategy, simple_grounded_answer, summarize_document_evidence, synthesize_multi_document_answer, format_citations, and update_memory (rag_service.py:143-213).

6. Document Ingestion Deep Dive

Input → function → processing → output → next component

1. Streamlit st.file_uploader in ui/sections.py:61-70 provides UploadedFile objects. On **Build Knowledge Base**, the UI forms UploadPayload(file_name, content) (:117-125).
2. KnowledgeBaseIngestionPipeline.ingest_uploads (services/indexing_pipeline.py:28) ensures storage, calls UploadManager.save_files, and immediately records successful save results in uploaded_files.json.
3. UploadManager.validate (upload_manager.py:24-42) allows only configured .pdf, .txt, .csv, nonempty input, and at most 50 MB. save_file (:64-112) SHA-256 hashes bytes, saves with Path.write_bytes under settings.paths.upload_dir, and builds DocumentMetadata. Current exact raw location: D:\hcl\data\uploads\<stored-name>; setting: PathSettings.upload_dir, built in config._build_paths.
4. Duplicate content is skipped by checksum against manifest (save_files, find_upload_by_checksum), regardless of name. Same name but different bytes follows configured rename/overwrite/error; observed strategy is rename. This is a content duplicate policy, not document-version management.
5. DocumentLoader.load_files delegates by suffix. PDF uses pypdf.PdfReader, processes every page independently, calls page.extract_text(extraction_mode="layout") with a plain-extraction fallback, normalizes

whitespace and likely table rows, and emits one `ExtractedDocument` per nonempty page (`loaders/pdf_loader.py`). Empty pages are skipped; a PDF with no usable page text fails. It is page-aware, but has no OCR for image scans and no explicit repeated-header/footer removal.

6. TXT uses UTF-8 with replacement for decoding errors and emits one `ExtractedDocument` (`txt_loader.py`). CSV uses `pandas.read_csv(dtype=str).fillna("")`, converts each nonempty row into column: `value | ...`, and emits one `ExtractedDocument` per row (`csv_loader.py`).

7. `ChunkingService.chunk_documents` (`chunking.py:34-65`) creates `ChunkRecord(text, ChunkMetadata)`. Chunk id format is `file_name:page_or_na:row_or_na:chunk_number` (:67-70). Important nuance: `chunk_number` restarts for **each extracted page/row/document**, not globally for the physical file. Page/row plus filename makes the persisted id usable.

8. Chunking first detects headings, prose/bullets/tables and retains a current `section_title`; headings are not included in block content. It combines semantic blocks up to the configured size; oversized blocks use `RecursiveCharacterTextSplitter` with separators `\n\n, \n, . , space`, empty string. Overlap is selected from prior non-table blocks up to 200 characters. Empty split output is discarded. No minimum-size merge rule exists; a small final chunk can remain. Table blocks are kept separate from prose and non-table overlap stops at table boundaries. Figure captions are neither specially parsed nor removed at ingestion.

9. `EmbeddingService.embed_texts` lazily loads `SentenceTransformer('all-MiniLM-L6-v2')` and calls `encode(..., batch_size=32, normalize_embeddings=True)`. Vectors are `float32`; observed dimension is 384. It makes no remote embedding call.

10. `VectorStoreService.add_embeddings` loads existing storage or creates `faiss.IndexFlatIP(dimension)`, adds vectors in order, builds parallel metadata records with `vector_id=start_index+offset`, then writes `index.faiss` and pickles state to `index.pkl` (`vector_store.py:47-93`).

There is no fixed maximum chunks-per-document: it depends on extracted text and layout. The observed corpus has 9,351 chunks across the four PDFs. There is no separately persisted extracted-text file or document id field; the raw file, filename/checksum manifest entry, and chunk metadata are the available linkage.

Failure, restart, deletion, and re-indexing facts

- Restart: `VectorStoreService.load_existing_index_state` reads both persistence files. It does **not** regenerate embeddings on every restart.
- Re-indexing: selecting Build Knowledge Base adds newly uploaded, unique content to the existing index. There is no “rebuild all existing uploads” command.
- Deletion: no per-document deletion/update API exists. `KnowledgeBaseService.clear` deletes everything within `uploads`, `faiss`, and `metadata`, recreates directories and an empty manifest. The UI exposes that all-or-nothing action.
- Partial indexing failure: upload manifest is written **before** extraction/embedding/index persistence. If later extraction fails, the raw file and its manifest entry remain. If FAISS persistence fails after `faiss.write_index` but before pickle write, files may be inconsistent; reads detect some inconsistency (`records < index.ntotal`) but no transaction, rollback, or recovery exists. This is a real limitation.

7. Exact Storage Architecture

Concept	Exact path / format	writer → reader	Actual meaning
Raw document	<code>data/uploads/<stored file></code> ; original format	<code>UploadManager.save_file</code> → loaders/opening by source path	Original bytes, e.g. the four PDFs.

Concept	Exact path / format	writer → reader	Actual meaning
Extracted text	No standalone file; in-memory ExtractedDocument.text	loader → chunker	Page text, TXT text, or CSV row text only during ingestion.
Chunk	data/faiss/index.pkl; pickle state['records'][i]['text']	MetadataManager.build_vector_from_text → RetrieverService._build_result	Text passed to LLM. context/LLM.
Metadata	data/faiss/index.pkl, plus upload manifest JSON	MetadataManager → retriever/UI	Chunk fields and vector mapping; upload checksum/provenance in JSON.
Embedding	FAISS internal float vectors in data/faiss/index.faiss	index.add → index.search	384-d normalized vector; not a PDF or text.
FAISS index	data/faiss/index.faiss; FAISS binary	faiss.write_index → faiss.read_index	IndexFlatIP, 9,351 rows during inspection.
Upload manifest	data/metadata/uploaded_files JSON list	record_uploads → duplicate checks/sidebar	Name/path/size/ext/SHA-256/timestamp/status
Memory	Streamlit st.session_state['conversation_memory'] in-memory	ConversationMemoryService ask_knowledge_base	Per browser/server session only, not file/database.
UI transcript/latest answer	Streamlit session-state keys	presenter/sections	Presentation/debug data only.
Prompts	prompts/*.txt; UTF-8	PromptManager → selected LLM service	Instructions, not user knowledge.
Logs	logs/product_support_agent.log; rotating UTF-8 text	logger → file/console	5 MiB × 5 backups configured.
Configuration	.env (secret excluded) + config.py defaults	dotenv → Settings	Runtime selection/limits/paths.

Vector-to-chunk mapping: IndexFlatIP returns zero-based row positions; index.pkl['records'][vector_index] is read by RetrieverService.retrieve. metadata.vector_id is written as the same starting position plus offset. This works because the code appends FAISS vectors and records in identical order. FAISS does not store the original PDF or a Python ChunkRecord; the chunk text and metadata live in pickle. The observed first record is MACHINE LEARNING(R17A0534).pdf:1:na:1, vector id 0, page 1.

8. User Question: Exact Runtime Trace

Input → function → processing → output → next component

1. UI: st.text_area and form in render_rag_section (ui/sections.py:210-230). The user string is appended to UI transcript and supplied to RAGService.answer_question(query, top_k=sidebar Search Depth).
2. Validation: RetrieverService.validate_query normalizes whitespace, rejects empty strings and >500 characters. Sidebar search depth defaults to 4, but a user may choose 1–20.

3. State: RAGService creates a RAGGraphState; it adds rag_context_expansion_chunks=2 to requested depth, so the default retrieval request is 6, before retriever candidate expansion.
4. Memory: load_memory obtains ConversationMemoryService.get_recent_history(). It retains user messages and only grounded assistant messages, returning the last 6 **messages** (not six turns). Memory is for reference resolution, never evidence.
5. Contextualization: with memory, QueryContextualizer.resolve_query calls the active LLM with query_contextualization_prompt.txt, temperature 0.0, max 512. It returns a cleaned standalone retrieval query or silently falls back to raw query on provider/contextualization failure. Without history it does not call the LLM.
6. Retrieval: RetrieverService.retrieve(resolved_query, top_k=6) embeds query locally with the same normalized embedding model. It requests top_k × retrieval_candidate_pool_multiplier; default multiplier is 5, hence 30 FAISS candidates for default RAG, constrained by index size.
7. FAISS: IndexFlatIP.search returns inner-product scores and positions. Because both documents/query are normalized, score is cosine similarity in practice. Each position indexes the parallel pickled record.
8. Reranking: lexical query terms are matched against filename, section title, and content. Score = semantic score + term-overlap ratio × 0.18 + section-title overlap × 0.06 + exact whole-query phrase bonus 0.12, minus structural penalty up to 0.20. This is not BM25 or a learned reranker.
9. Retrieval quality: exact duplicate text within a source file is removed; adjacent chunks are expanded within the same file/page/row keyed by chunk number. A structural anchor scans at least four neighbours, ordinary anchors use configured window 1; neighbours must be nonstructural, ≥90 normalized characters, and ≥12 alphabetic tokens. Result is sliced to requested top-K.
10. Relevance gate: graph filters on rerank_score if present, otherwise similarity, requiring ≥0.35. If none remain, it returns the fixed insufficient-evidence sentence and never calls the answer LLM.
11. Strategy: if relevant chunks come from >1 file **and** raw query contains a signal such as compare, summary, across, both, versus, or vs, it selects MULTI_DOCUMENT_REASONING; otherwise SIMPLE_QA. Mere multi-file retrieval is not enough.
12. Context/prompt: simple mode formats [SOURCE n], file/type/page/row/chunk/id and content (RAGService._build_context). It loads system_prompt.txt and calls configured provider. Multi-document mode groups retrieved chunks per filename, calls the LLM once per document with document_summary_prompt.txt, then calls it again with multi_document_synthesis_prompt.txt.
13. Citations: format_citations creates SourceReference objects from retrieval metadata in Python; the model does not invent citation objects. The UI shows source location and preview.
14. Update: update_memory writes original user query and nonempty assistant result into session memory; ungrounded assistant responses are stored but excluded from future recent history. UI records elapsed wall time around whole answer_question call.

9. Chunking, Embeddings, and FAISS Deep Dive

Chunking defence

Chunking bounds retrieval and prompt context: an entire manual is too broad to embed/retrieve as one unit. Fixed chunks are simple but can split sentences; recursive chunks try increasingly smaller natural separators; sentence/paragraph/semantic/section-aware approaches preserve more meaning but add heuristics or cost. This implementation is **structure-aware recursive character chunking**, not semantic embeddings-based chunking or parent-child retrieval. 1,000 characters/200 overlap are configuration values, not a universal optimum; the trade-off is fewer broader chunks and duplicated boundary text versus more precise but fragmented chunks.

Headers become section_title when heuristics detect a colon-ending heading, short uppercase line, or short uppercase pipe-delimited heading. This helps metadata and lexical boosts but has false-positive/false-negative risk. Repeated headers, captions and table-of-contents references are not removed at ingestion. Tables are normalized by PDF loader and treated as table blocks; they are not structurally reconstructed. Captions may be indexed, then penalized at

retrieval if they look like figure captions. Page boundaries are preserved because PDF pages become separate extracted documents; chunks do not cross pages.

The chapter-header retrieval problem

The current source changes explicitly address a historical/observed retrieval pattern in tests: chapter/contents/figure structure looked semantically similar to a chapter-specific query. Embeddings encode related words such as the chapter topic and number; FAISS Top-K only ranks similarity, it does not know that a header lacks explanatory content. Top-K therefore allows a header through. Current code adds structural penalties (chapter header -0.14, contents -0.08, figure caption -0.06 or -0.20, capped -0.20), deduplicates repeated text, and expands nearby substantive chunks. The neighbour expansion is helpful context recovery, not proof of relevance; it remains subject to final top-K and relevance filtering. Future **recommended** ingestion improvements include page-header/footer detection, richer section hierarchy, and evaluated re-indexing—but do not weaken the grounder: better evidence selection is the correct remedy.

Embeddings and FAISS defence

An embedding is a dense numeric representation of text meaning. Here it represents a chunk (or query), not the document's bytes, author intent, or a guaranteed factual relation. `all-MiniLM-L6-v2` produces observed 384-float vectors, normalized to unit length. `IndexFlatIP` computes exact inner-product nearest neighbours; normalized inner product corresponds to cosine similarity. `IndexFlatIP` has no approximate compression/training and is appropriate for the present 9,351-vector local prototype, but memory and exact-search latency grow linearly with vector count.

Top-K means the requested number of final retrieved context items, after candidate expansion/reranking logic. Similar scores mean the query vector is near a chunk vector; they are **not** probability, correctness, truth, or a calibrated relevance guarantee. Two semantically similar chunks both can rank; exact-text deduplication only removes same-file identical normalized content. A heading can outrank body text because its short words closely align with query and vector similarity lacks an understanding of informational density—hence the added penalties.

10. Query Contextualization and Conversation Memory

Example: after “What is Generative AI?”, “What are its applications?” is presented with recent user/grounded-assistant messages to the contextualizer, which may return “What are the applications of Generative AI?” That rewritten text is embedded/retrieved; final generation sees the original question plus resolved query and fresh retrieved text. For “Summarize the chapter discussed earlier,” the prompt asks it to resolve the subject only if history establishes one; otherwise code falls back to unchanged ambiguous text. Memory is not persisted, is lost on session/server restart, caps storage at 12 messages by default, and recent context at 6 filtered messages. Risks: ambiguity, a bad rewrite, long-dialogue truncation, unrelated retained user questions, and context pollution. The fallback protects retrieval from an empty rewrite but cannot prove a rewrite is correct.

11. Grounding / Anti-Hallucination

Grounding means answer content is constrained to retrieved document evidence. Enforcement is layered:

- `prompts/system_prompt.txt` says “ONLY” retrieved context, no outside knowledge, and says memory is not factual evidence.
- `RAGService._node_filter_relevance` refuses scores below 0.35 before generation.
- Empty results, low relevance, unavailable/missing index, empty context, and exceptions route to safe fallback instead of an answer LLM (`rag_service.py:231-302, 648-676`).
- `PromptManager` uses similarly grounded instructions for per-document summaries and synthesis.
- Citations are deterministic Python metadata, not generated filenames/pages.

Retrieval failure means no appropriate/usable evidence was selected; generation failure means a provider call failed/returned empty/incomplete after evidence was selected. The former yields the fixed no-evidence sentence; the latter returns an empty answer with “Grounded answer generation is currently unavailable.” Prompt instructions are safeguards, not formal proof against an adversarial/misbehaving model. Improving retrieval and evaluating faithfulness is preferable to allowing unsupported best-effort answers.

12. Multi-Document Reasoning and Citations

Implemented sequence: retrieve all candidates globally; retain relevant chunks; group by `source_file`; only choose multi-document mode when multiple files and query keywords indicate comparative/synthesis intent; summarize each file's retrieved chunks; synthesize summaries; cite the original retrieval metadata. It does not identify product/version independently, force document coverage, resolve conflicts, or send all documents. A question about E104 across an installation guide and known-issues document will take this path only if both files are retrieved and the wording has a listed multi-document signal. Conflicting evidence is currently passed to the model without a deterministic “newest version wins” policy. At 100 documents, retrieve/filter first; never send every document to the LLM. Add product/version metadata filters and evaluation before claiming reliable version-aware reasoning.

13. Technology Decisions: Problem → Choice → Alternatives → Trade-off

Technology actually used	Why appropriate here	Alternatives / replacement trigger
Python	Fast modular prototype ecosystem for parsing, ML and Streamlit.	Java/TS services for enterprise platform; replace only for team/platform needs.
Streamlit	Direct local upload/chat/admin UI with session state.	React + API for auth, multi-user workflows, scale.
PyPDF	Lightweight page-aware extractable-text PDF reader.	PyMuPDF/pdfplumber/OCR pipeline when layout/OCR quality is insufficient.
pandas CSV	Simple row-level normalization.	CSV module/ETL schema validation for very large or sensitive structured data.
Sentence Transformers MiniLM	Local, normalized semantic vectors without an embedding API.	Larger/domain model after retrieval evaluation; any model change requires full re-index.
FAISS IndexFlatIP	Exact local cosine-style search, easy persistence, suitable for 9,351 vectors.	HNSW/IVF FAISS for larger local collections; Qdrant/Weaviate/Pinecone for distributed filtered multi-tenant scale.
LangGraph	Explicit observable routing and fallback nodes around custom services.	A simple function chain if no branching; it is not autonomous agency.
Ollama/qwen3:8b (active)	Local endpoint supports a privacy-oriented demo without cloud answer generation.	Hosted Gemini is supported; production model choice needs quality/cost/security evaluation.
Gemini (optional)	Existing SDK adapter keeps cloud provider option.	Not active in observed <code>.env</code> .

Technology actually used	Why appropriate here	Alternatives / replacement trigger
Pickle metadata + filesystem	Minimal parallel text/metadata persistence.	SQLite/Postgres/object store for transactions, deletion, querying and safety.
Streamlit session memory	Zero infrastructure and session isolation.	Redis/database when durable/concurrent history is required.
Python logging	Basic operational record with rotation.	Structured logs/metrics/traces for production observability.

14. RAG Approaches Compared to This Project

Approach	Does this project use it?	Value / limitation
LLM-only	No.	Faster to build, but cannot audit current documents.
Fine-tuning	No.	Changes model behaviour; poor fit for frequently updated manuals and not a citation store.
Keyword/BM25	No.	Strong for exact error codes; current lexical boosts are not BM25. Recommended hybrid candidate retrieval later.
Dense/vector retrieval	Yes.	Semantic match via MiniLM + FAISS; may retrieve structural semantically similar text.
Hybrid retrieval	No.	Recommended if error codes/version strings prove weak under dense retrieval.
Basic RAG	Yes.	Retrieve chunks then prompt LLM.
Advanced retrieval quality	Partly.	Candidate pool, lexical boosts, structural penalties, dedupe, adjacency; no learned reranker.
Conversational RAG	Yes.	Query rewrite from session memory, still grounded in retrieval.
Multi-document RAG	Conditional.	Per-document summaries + synthesis for selected comparative queries.
Agentic RAG	Limited.	LangGraph routes deterministic nodes; no autonomous planning/tool loop.
Metadata-aware filtering	Partial only.	Metadata is preserved/displayed; no product/version/filter query API.

Approach	Does this project use it?	Value / limitation
Reranking	Heuristic only.	No cross-encoder.
Parent-child/hierarchical/multi-query	No.	Recommended only after evidence shows a need.

15. Performance and Scalability

No formal benchmark harness/targets are implemented; current measured UI response time is only end-to-end `perf_counter` shown in session debug. Likely bottlenecks are PDF parsing (especially 801-page file), CPU embedding generation, exact `IndexFlatIP` candidate search, one contextualization LLM call when history exists, final LLM generation, and one extra LLM call per selected document in multi-document mode. Chunking itself is comparatively light.

Scale	Current architecture judgement	Preserve grounding; recommended evolution when justified
10 documents	Appropriate.	Keep current local stack.
100 documents	Often workable depending on chunk count/model hardware.	Add evaluated metadata/filter schema; batch/background ingestion.
1,000 documents	Exact flat search and pickle lifecycle become operationally awkward.	ANN FAISS index, durable metadata DB, hybrid retrieval/reranker after evaluation.
10,000 documents	Prototype lifecycle/deletion/concurrency insufficient.	Service/API separation, queues, object storage, filtered vector DB.
100,000+	Not appropriate as-is.	Distributed/vector DB, tenant isolation, observability, model serving, access control, DR.

Safe performance optimizations: reuse loaded embedding/LLM objects in a longer-lived service, batch ingestion, cache only safe deterministic results, reduce LLM calls only after quality evaluation, pre-filter by product/version once that metadata exists, and use ANN only with recall tests. Do not lower relevance thresholds or remove refusal solely for speed.

16. Security Assessment

Implemented: extension/nonempty/size validation, basename use for upload name, SHA-256 duplicate detection, local raw/vector storage, no raw `source_path` placed in the constructed model context, and logs. Missing: authentication, authorization, document ACLs, encryption, malware scanning, MIME/content validation, rate limits, audit identities, product/version/tenant isolation, secret management beyond `.env`, retention policy, prompt-injection hardening for retrieved documents, dependency pinning, and production monitoring. A document can contain instructions that influence the generation model because retrieved chunk text enters the prompt; the system prompt helps but is not a robust content-security boundary. Treat uploaded documents and logs as potentially sensitive.

17. Tests, Evaluation, and Traceability

The repository has 62 static pytest test functions. They cover loaders, chunking, embedding invocation, metadata, FAISS persistence/restart, upload duplicate logic, retriever validation/rerank/adjacency, memory, KB clear, Gemini behaviours, contextualizer, and RAG graph outcomes. No tests were executed for this forensic report to honour the “do not modify project” instruction. There are no Ollama provider tests found, no end-to-end test against the current sample corpus, no benchmark, no evaluation dataset, and no production security test.

Evaluation case	Requirement/component → evidence to test	Expected result
Direct fact	Retriever + RAG simple path	Correct cited answer from gold chunk.
Contextual follow-up	Memory + contextualizer	Standalone resolved query retrieves intended evidence.
Specific chapter	Structural penalties/adjacency	Substantive chapter chunk outranks header or is included nearby.
Summary	Retrieval + RAG	Scope-limited cited summary, no unsupported claims.
Multi-document	Strategy + summaries/synthesis	Both gold documents selected and citations trace back.
Source-specific	Metadata/citations	Correct filename/page/row/chunk.
Insufficient evidence	Relevance gate	Refusal, no fabricated answer.
Obsolete version	Not implemented version metadata	Must be a known failure until metadata filters exist.
Error code	Dense/lexical evaluation	Measure recall; consider hybrid if weak.
Repeated headers	Dedupe/penalty test	Header does not displace substantive evidence.

Core traceability: upload FR → UploadManager.validate/save_file, test_upload_manager.py; extraction → loaders, test_document_loaders.py; chunks → ChunkingService, test_chunking.py; vectors → EmbeddingService/VectorStoreService, their tests; retrieval quality → RetrieverService, test_retriever.py; grounding/citations/LangGraph → RAGService, test_rag_service.py; memory → ConversationMemoryService, test_memory_service.py; clear → KnowledgeBaseService, test_knowledge_base_service.py.

18. Documentation-versus-Code Review

Artefact / claim	Classification	Finding
README says “Gemini-powered” and Gemini setup is minimum	Documentation stale/incomplete	Current .env and provider factory use Ollama/qwen3:8b. README needs a future review, but this report did not edit it.

Artefact / claim	Classification	Finding
README project tree	Documentation incomplete	Omits several active newer modules: loaders, ingestion, KB lifecycle, embedding, LLM package and much UI.
SRS/HLD/LLD/UID current Markdown	Implemented/aligned in major claims	They describe Ollama active, LangGraph, retrieval controls, current paths; code supports those.
"memory_max_turns" described as turns	Documentation wording mismatch	Code slices messages ([-memory_max_turns :]) and stores at most 2x; it is not precisely turns.
"multi-document reasoning" generic promise	Implemented with condition	Only activated if >1 relevant source and one keyword signal occurs.
requirements.txt	Code/dependency mismatch	Active Ollama provider imports ollama, but requirements does not declare it. It will fail safely if package absent.
Gemini libraries / GeminiService	Implemented optional/legacy-adaptor path	Retained for provider selection and tests; inactive under observed configuration.
Product positioning	Documentation/product claim only	Current indexed corpus is academic/sample PDFs, not real product documents.

19. Current Limitations and Recommended but Not Yet Implemented

- **Document lifecycle:** no per-document delete, replace, checksum-version relationship, rebuild, or atomic ingest transaction.
- **Security/tenancy:** no auth/ACL/tenant/product/version filtering.
- **Retrieval:** no BM25/hybrid search, learned reranker, calibrated thresholds, query filters, or corpus evaluation.
- **Parsing:** no OCR and no robust header/footer/table/figure semantic extraction.
- **Scale:** flat index + pickle is single-process/local persistence, not concurrent transactional storage.
- **Generation:** provider availability/model installation is not guaranteed; Ollama package omission from requirements is an operational gap.
- **Observability:** logs but no structured tracing, metrics, alerting, or retrieval quality dashboards.

Recommended roadmap: first create a gold set from genuine product documents and evaluate retrieval/citation/refusal; then add immutable document/version metadata and filtered retrieval; add atomic ingestion/individual lifecycle; introduce hybrid retrieval or reranking only if gold metrics justify it; finally add authentication, authorization, malware scanning, durable stores, queues, and production observability. Grounding policy stays unchanged.

20. File-by-File Code Walkthrough Cards

File	60-second mentor explanation
<code>app.py</code>	"This is only composition: it makes <code>src</code> importable, loads typed settings, bootstraps directories/logging/manifest, and renders Streamlit."
<code>config.py</code>	"All runtime knobs and paths are centralized in immutable settings built from <code>.env</code> with defaults; it is the source of truth for chunking, retrieval, LLM and storage."
<code>ui/sections.py</code>	"This is the real UI-to-domain boundary: uploader creates byte payloads for the ingestion pipeline; chat constructs session memory and invokes <code>RAGService</code> ."
<code>upload_manager.py</code>	"It validates extension/size/content, uses SHA-256 for content duplicates, saves raw bytes, and returns metadata; it does not index."
<code>document_loader.py</code> and <code>loaders/*</code>	"Facade chooses PDF/TXT/CSV parser; PDFs are page-aware, TXT whole-file, CSV row-aware."
<code>chunking.py</code>	"It is a structure-aware chunker: detects headings/table blocks, preserves page/row metadata, uses 1,000/200 recursive splitting only for oversized text."
<code>embedding_service.py</code>	"It lazily loads MiniLM, batches text at 32, normalizes 384-d floats, and never calls a remote embedding API."
<code>metadata_manager.py</code>	"It owns two persistence forms: JSON upload manifest and pickle vector state containing the chunk text and metadata."
<code>vector_store.py</code>	"It persists exact FAISS inner-product vectors and keeps its row positions aligned to pickle records; it does not store PDFs/text in FAISS."
<code>indexing_pipeline.py</code>	"It coordinates <code>save</code> → <code>extract</code> → <code>chunk</code> → <code>embed</code> → <code>persist</code> . Its sequencing exposes the current partial-failure limitation."
<code>retriever.py</code>	"It embeds a query, searches a larger FAISS pool, adds lexical/section/exact boosts, structural penalties, dedupe and adjacency, then returns metadata-rich chunks."
<code>memory.py</code>	"It is filtered recent in-memory Streamlit session history, not a database and not evidence."
<code>query_contextualizer.py</code>	"It calls the selected LLM only to rewrite a follow-up into a standalone retrieval query; it falls back to raw query on failure."
<code>rag_service.py</code>	"This is the orchestration centre: explicit LangGraph nodes enforce retrieval/relevance before simple or conditional multi-doc generation and deterministic citations."
<code>services/llm/*</code> and <code>gemini_service.py</code>	"Provider contract selects active Ollama or optional Gemini adapter; each accepts prompt/context and validates provider responses."

File	60-second mentor explanation
<code>prompt_manager.py / prompts/*</code>	"They load grounding, rewrite, summary and synthesis instructions from files without putting factual knowledge in prompts."
<code>knowledge_base_service.py</code>	"It only supports destructive all-KB clear and recreates empty directories/manifest; individual document deletion is absent."
<code>logger.py</code>	"It configures console plus 5 MiB rotating UTF-8 file logs."

Component Contract and Failure Map

Component / key functions	Input → output	Direct dependencies/configuration	Failure behaviour / scale implication
<code>config.py</code> : <code>_build_paths</code> , <code>_build_app</code> , <code>ensure_runtime_directories</code>	OS env → Settings, directories	<code>python-dotenv</code> , project root <code>.env</code>	Invalid numeric values quietly use defaults; unexpected config can therefore be hidden.
<code>UploadManager</code> : <code>validate</code> , <code>save_files</code> , <code>save_file</code>	<code>UploadPayload[]</code> → <code>UploadResult[]</code> , raw files	Paths, extension/size/duplicate settings; <code>MetadataManager</code>	Raises validation error. Hash lookup is linear over manifest; no antivirus/MIME inspection.
<code>DocumentLoader</code> / loaders: <code>load_file</code> , <code>load</code>	saved path → <code>ExtractedDocument[]</code>	<code>pypdf</code> , <code>pandas</code> , extension routing	Per-file extraction errors are collected; unextractable PDF/TXT/CSV blocks that file. No OCR.
<code>ChunkingService</code> : <code>chunk_documents</code>	extracted units → <code>ChunkRecord[]</code>	LangChain splitter, chunk size/overlap	No output raises pipeline error. Character-based cost is light; hierarchy is heuristic, not a parser.
<code>EmbeddingService</code> : <code>embed_texts</code>	strings → normalized <code>np.float32[n, 384]</code>	Sentence Transformers, model/batch settings	Model/runtime errors map to <code>EmbeddingError</code> ; generation scales with total chunks.
<code>MetadataManager</code> : manifest/vector state methods	models/chunks → JSON/pickle and back	filesystem paths, json, pickle	Corruption raises <code>MetadataError</code> ; pickle is local trusted-state storage, not a safe untrusted format.
<code>VectorStoreService</code> : <code>add_embeddings</code> , <code>search</code>	vectors/query vector → persisted index/results/state	FAISS, NumPy, metadata manager	Dimension/inconsistency errors fail retrieval safely; writes are not atomic and FlatIP is O(N) query work.
<code>RetrieverService</code> : <code>retrieve</code>	resolved query → <code>RetrievalResponse</code>	embedding + vector store; K/candidate/boost/window knobs	Missing/corrupt index raises error. Metadata is used for display/adjacency, not query-time filters.

Component / key functions	Input → output	Direct dependencies/configuration	Failure behaviour / scale implication
ConversationMemoryService: Streamlit mapping → get/add history	ConversationMessage[]	session id, memory limit	No persistence or user auth; capacity is deliberately small but context can still be misleading.
QueryContextualizer: resolve_query	raw query + memory → retrieval query	selected LLM, rewrite prompt, temp/max tokens	Provider/error/empty rewrite falls back to original query, preserving availability but not resolving ambiguity.
RAGService: answer_question, graph nodes	raw question + K → RAGResponse	LangGraph, retriever, LLM, prompts, memory	Any unavailable evidence returns refusal; generation errors return unavailable state. Multi-doc LLM calls grow per retrieved file.
OllamaService / GeminiLLMAdapter	system instruction + constructed context → text	active provider/model, endpoint/API key	Validates empty/incomplete responses. Ollama package missing from requirements is a deploy risk.
KnowledgeBaseService: clear	clear request → deletion counts	upload/faiss/metadata paths	Logs file-level failures, but no undo, per-document operation, or transactional recovery.
ui/sections.py / presenter	Streamlit widgets/session → service calls and display	Streamlit, services, UI keys	Catches domain errors and displays them. It has no authentication/rate limiting/API boundary.

Important import relationships. `indexing_pipeline.py` imports and composes upload, loader, chunking, embedding, metadata, and vector services. `retriever.py` imports embedding/vector/metadata. `rag_service.py` imports retriever, provider factory, prompts, contextualizer, memory and LangGraph lazily inside `_build_graph`. `ui/sections.py` is the only production caller of ingestion, KB clear and RAG public operations found by repository search. All service classes use `config.Settings` and `logger.get_logger`; schemas are their interchange types. Tests create these classes directly with fixture settings and fakes.

Execution-order detail for mentor walkthrough. The request should not be described as “UI → LLM.” Ingestion order is specifically save/manifest → parse → chunk → embed → index/pickle; question order is validate → memory → rewrite → local query embed → FAISS → rerank/adjacency → relevance gate → provider → metadata citations → memory. This ordering is the architectural proof that retrieval is factual authority and the LLM is a constrained synthesizer.

21. Mentor Defence Question Bank (100)

Format: **Short answer** | **Detail / proof** | **Why asked** | **Common wrong answer** | **Strong follow-up**.

- What is this project?** | Grounded local RAG support prototype | `app.py`, `RAGService` | “A chatbot” | “It is only product-ready after real-doc evaluation/security.”
- Who uses it?** | Admin uploads; support user asks | `ui/sections.py` | “Any authenticated user” | “No authentication exists.”

3. **What business problem?** | Find approved-document answers with audit trail | prompts/citations | “Replace support engineers” | “It assists and cites; it does not decide.”
4. **Are real product manuals indexed?** | No; four academic/sample PDFs observed | data/uploads | “Yes” | “Validation with genuine corpus is next.”
5. **UI entry?** | Streamlit app.py → layout | app.py:main | “FastAPI endpoint” | “No HTTP API service is present.”
6. **Upload formats?** | PDF, TXT, CSV | config.py, UploadManager | “All files” | “Extension, empty content and 50 MB checked.”
7. **Where raw upload stored?** | data/uploads/<stored-name> | UploadManager.save_file | “FAISS” | “FAISS never stores raw PDF.”
8. **Who saves it?** | Path.write_bytes in UploadManager | upload_manager.py | “Streamlit automatically” | “Streamlit supplies bytes only.”
9. **Duplicate policy?** | SHA-256 content skip | save_files | “Same filename only” | “Different names/same bytes also skip.”
10. **Same name/different content?** | Rename by observed configuration | _resolve_target_path | “Always overwrite” | “Overwrite/error are configurable alternatives.”
11. **PDF parser?** | pypdf.PdfReader | pdf_loader.py | “LangChain loader” | “It calls page extraction directly.”
12. **PDF page awareness?** | Yes, one ExtractedDocument/page | PDFDocumentLoader.load | “No” | “Page number propagates to citation.”
13. **Scanned PDF support?** | No OCR implemented | loader code | “PyPDF OCRs” | “No text means extraction failure.”
14. **TXT parsing?** | UTF-8, replacement errors, one extracted doc | txt_loader.py | “Line-by-line” | “No page/row metadata.”
15. **CSV parsing?** | pandas rows into column: value text | csv_loader.py | “One giant string” | “Row number becomes metadata.”
16. **Extracted text persisted?** | No standalone artifact | storage map | “In JSON” | “It becomes pickled chunk text after chunking.”
17. **Chunk object?** | ChunkRecord + ChunkMetadata | schemas.py | “LangChain Document” | “Custom dataclass used.”
18. **Chunk ID?** | filename:page:row:chunk | _build_chunk_id | “UUID” | “Chunk number is local to extracted unit.”
19. **Chunk size?** | 1,000 chars | .env, config | “1,000 tokens” | “Length function is Python len.”
20. **Overlap?** | 200 chars/non-table semantic blocks | chunking.py | “Always exact 200” | “Up to threshold, semantic blocks only.”
21. **Splitter?** | RecursiveCharacterTextSplitter fallback | chunking.py | “Sentence transformer” | “Separators preserve natural boundaries.”
22. **Why chunk?** | Retrieval/prompt precision and bounded context | architecture | “FAISS needs it” | “It also supports citations.”
23. **Heading handling?** | Heuristic section metadata, omitted from content blocks | _looks_like_heading | “Full outline parser” | “Heuristics can misclassify.”
24. **Header/footer handling?** | No robust removal | source inspection | “Removed” | “Retrieval penalties mitigate some effects.”
25. **Tables?** | Normalized then kept as table blocks | PDF/chunker | “Perfectly structured” | “No table model/schema.”
26. **Figures/captions?** | Not parsed specially; captions can be penalized | retriever | “Ignored” | “May still be indexed.”
27. **Empty chunks?** | Discarded after split/merge | if text_chunk.strip() | “Embedded” | “Empty source may fail earlier.”
28. **Small chunks?** | Can remain | chunker | “Merged always” | “Only adjacency has substantive minimum.”
29. **Embedding model?** | all-MiniLM-L6-v2 | observed .env | “Qwen” | “Qwen is answer model.”
30. **Embedding location?** | EmbeddingService._get_model lazy local load | service | “FAISS loads it” | “FAISS consumes vectors only.”

31. **Vector dimensions?** | 384 observed | `index.pkl`, FAISS | "1,000" | "Chunk size is unrelated."
32. **Normalization?** | Yes `normalize_embeddings=True` | embedding service | "Maybe" | "Inner product becomes cosine-like."
33. **FAISS type?** | `IndexFlatIP` exact inner product | vector store | "HNSW" | "Flat search scales linearly."
34. **Does FAISS store PDF?** | No | FAISS/vector state | "Yes" | "Raw bytes remain uploads."
35. **Does FAISS store chunk text?** | No | `index.add` | "Yes" | "Pickle records store it."
36. **Where are chunks?** | `data/faiss/index.pkl` records | `MetadataManager` | "upload manifest" | "Manifest has upload metadata only."
37. **Vector-to-text mapping?** | FAISS row index → same pickle record position | retriever | "Magic metadata search" | "Append order is the contract."
38. **What is vector_id?** | Parallel zero-based insertion ID | build records | "Random ID" | "No FAISS IDMap is used."
39. **Restart behaviour?** | Reloads index+pickle; no re-embed | vector store | "Rebuilds everything" | "Dimension mismatch fails."
40. **Model change?** | Existing dimension/model state incompatible/semantically stale | vector store | "Works automatically" | "Clear/rebuild required."
41. **Re-index process?** | Add new unique uploads | pipeline | "Full rescan" | "No rebuild-all command."
42. **Document delete?** | Not implemented individually | KB service | "Remove vector" | "Only clear all KB."
43. **Clear KB?** | Deletes uploads/faiss/metadata contents, recreates manifest | KB service | "Deletes source code" | "UI also clears session conversation."
44. **Partial indexing failure?** | Possible orphan manifest/raw and non-atomic index pair | pipeline/vector store | "Transaction rollback" | "Needs future atomic lifecycle."
45. **Question validation?** | Normalize/reject empty/>500 chars | Retriever | "LLM validates it" | "Validation occurs before embedding."
46. **Query rewrite?** | LLM standalone rewrite with history | contextualizer | "Model answer" | "It does not supply facts."
47. **No memory case?** | Raw normalized query passes through | contextualizer | "Still calls LLM" | "Avoids unnecessary latency."
48. **Memory storage?** | Streamlit session state | memory service | "FAISS" | "Lost after session restart."
49. **Memory evidence?** | Never factual authority | system prompt | "It is context proof" | "Only retrieval grounds answer."
50. **Memory length?** | Last six filtered messages, max twelve stored | memory service | "Six turns" | "Documentation wording is imprecise."
51. **Top-K?** | Final requested context count | retriever | "FAISS database limit" | "Candidate pool is larger first."
52. **Candidate pool?** | Top-K ×5 default | Retriever | "Always four" | "Default RAG asks 6 then candidates ~30."
53. **Similarity score?** | Normalized-vector inner product/cosine-like | FAISS | "Probability" | "Not calibrated truth."
54. **Reranking?** | Semantic plus lexical/title/exact heuristics | `_enrich_result` | "Cross encoder" | "No learned reranker."
55. **Why header ranks?** | Semantic term similarity lacks content-density understanding | retrieval | "FAISS bug" | "Penalties/adjacency target it."
56. **Structural penalty?** | Header/contents/figure heuristic subtraction | `_compute_structural_penalty` | "Deletes chunk" | "It changes rank only."
57. **Deduplication?** | Same file + normalized identical text | retriever | "All similar meanings" | "Only exact normalized fingerprints."
58. **Adjacent expansion?** | Adds substantive same page/row neighbours | retriever | "Across document" | "Not parent-child retrieval."
59. **Relevance threshold?** | ≥ 0.35 rerank/similarity | RAGService | "LLM confidence" | "Code threshold gate."
60. **No evidence?** | Fixed refusal, no answer generation | fallback node | "Guesses answer" | "Grounding retained."
61. **LLM active?** | Ollama qwen3:8b | observed `.env` | "Gemini" | "Gemini is optional."

62. **Ollama connection?** | UI probes `/api/tags`; provider chats locally | presenter/provider | “Guaranteed model loaded” | “Endpoint does not prove model installed.”
63. **Ollama dependency?** | Code imports it, requirements omits it | requirements/service | “Guaranteed installed” | “Operational gap.”
64. **Gemini role?** | Optional adapter/legacy service path | llm factory | “Dead code” | “Selectable if configured.”
65. **Prompt location?** | `prompts/*.txt` | PromptManager | “Hardcoded only” | “Defaults create missing files.”
66. **Grounding prompt?** | System prompt restricts to retrieved blocks | `system_prompt.txt` | “Citation alone grounds” | “Control flow also gates.”
67. **Citation generator?** | Python `SourceReference` from metadata | RAGService | “LLM text” | “Auditable deterministic objects.”
68. **Citation fields?** | file/page/row/chunk/id | schemas | “URL” | “No URLs generated.”
69. **LangGraph purpose?** | Explicit conditional orchestration | `_build_graph` | “Autonomous agent” | “Deterministic state graph.”
70. **Graph start?** | load memory → contextualize → retrieve | RAGService | “LLM first” | “Retrieval precedes answer.”
71. **Simple strategy?** | One-file/default direct grounded answer | strategy function | “Always” | “Multi path conditional.”
72. **Multi-doc trigger?** | >1 files plus keyword signal | strategy function | “Any two documents” | “Signal list is limited.”
73. **Multi-doc process?** | Per-file summary then synthesis | RAGService | “All docs prompt” | “Only retrieved chunks.”
74. **Conflict policy?** | None deterministic | source inspection | “Latest wins” | “Need version metadata/policy.”
75. **Is it agentic?** | Orchestrated RAG in loose sense, not autonomous agent | graph code | “Yes, tools plan themselves” | “No tool loop/planning.”
76. **Logs?** | Rotating local file + console | logger | “Database audit trail” | “No structured telemetry.”
77. **Latency measure?** | Whole UI answer milliseconds | sections | “Per-stage metrics” | “Per-stage instrumentation absent.”
78. **Likely slow stage?** | Local generation/PDF/embedding/flat search | code architecture | “FAISS only” | “Multi-doc adds calls.”
79. **Why FAISS not Pinecone?** | Local/no service appropriate now | scale table | “FAISS best” | “Pinecone/vector DB later for ops/filters.”
80. **Why no BM25?** | Not implemented | Retriever | “Lexical boost is BM25” | “Hybrid is a future evaluation decision.”
81. **100 docs?** | Likely workable but evaluate count/hardware | scalability | “Guaranteed” | “Add metadata and batch ingestion.”
82. **100k docs?** | Not suitable as-is | scale table | “Just increase Top-K” | “Needs distributed lifecycle/ANN/ACL.”
83. **Caching?** | No explicit query/embedding cache | source inspection | “FAISS is cache” | “Model instances lazy per service only.”
84. **Concurrency?** | No locks/transaction design | filesystem code | “Streamlit solves it” | “Concurrent writes risk corruption.”
85. **Authentication?** | None | UI/source inspection | “Streamlit login” | “Production blocker.”
86. **Authorization?** | None | source inspection | “Metadata protects docs” | “Metadata is not an ACL.”
87. **Tenant isolation?** | None | storage paths | “Separate chunks automatically” | “All corpus is globally searchable.”
88. **Prompt injection?** | No robust retrieved-content defence | prompts | “System prompt prevents all” | “Treat docs untrusted; add controls.”
89. **Malicious files?** | Extension/size only | UploadManager | “Virus scanned” | “Need scanning/content validation.”
90. **Secrets?** | `.env` loaded, no secret manager | config | “Safe in repo” | “Do not expose API keys/log prompts.”
91. **Tests count?** | 62 static pytest tests | `tests/` | “Fully production tested” | “No corpus evaluation/Ollama test.”
92. **Retrieval recall metric?** | Not implemented yet | source inspection | “Similarity score” | “Build gold question→chunk set.”

93. **Citation correctness proof?** | Metadata mapping plus tests; needs manual gold audit | RAGService/tests | "LLM cited it" | "Trace row index to pickle record."
94. **Refusal accuracy?** | Evaluate in/out-of-scope question set | fallback code | "Always good" | "Threshold needs calibration."
95. **Version correctness?** | Not implemented | schemas | "Filename contains version" | "Add first-class metadata/filter."
96. **Why sample demo weak?** | Business data/model does not match claimed support use | observed corpus | "Architecture is enough" | "Validate with product docs/scenarios."
97. **Main retrieval fix?** | Penalty/dedupe/adjacency; no reindex performed here | git/source diff | "Changed grounding" | "Grounding unchanged."
98. **What proves persistence?** | index files and restart tests | VectorStore/tests | "Memory state" | "Files reload after process restart."
99. **Production next step?** | Gold evaluation then metadata/lifecycle/security | roadmap | "Replace FAISS first" | "Solve measured needs in order."
100. **Biggest honest limitation?** | Product claim exceeds current sample-data/security/lifecycle validation | report evidence | "None" | "State limits with remediation plan."

22. Client Presentation Scripts

60 seconds

"This is a document-grounded support-assistant prototype. An administrator uploads PDF, TXT, or CSV files; the system keeps the original local file, extracts page or row text, makes metadata-rich chunks, creates local MiniLM embeddings, and persists an exact FAISS index plus a parallel metadata store. For a question, it may rewrite a follow-up using session memory, retrieves and quality-filters evidence, then lets the selected LLM answer only from that context. Citations come from retrieval metadata in Python, not from model invention. The active demo uses local Ollama qwen3:8b, but the current data are sample academic PDFs, not actual product manuals. The next validation step is a genuine product corpus with retrieval and citation evaluation."

5 minutes

Explain the same flow in five beats: business need (grounded support), ingestion and storage distinction (raw file vs chunk text vs vector), retrieval quality controls (candidate pool, lexical boosts, structural penalties, adjacency), LangGraph grounding gates (no/weak evidence refuses), and honest production gap (no ACL/versioning/atomic lifecycle or real-product evaluation). Show the Developer Tools panel with resolved query, graph nodes, scores and source previews; then open `index.pkl` conceptually as the vector-to-text mapping.

15 minutes technical explanation

Walk `app.py` → UI → `KnowledgeBaseIngestionPipeline`, then inspect one `ChunkRecord` and one pickle record. Explain 384-d normalized MiniLM vectors and `IndexFlatIP`. Walk the graph nodes and point out the 0.35 relevance gate before LLM. Demonstrate contextual follow-up, one simple answer, a comparative question satisfying the multi-doc signal, and an insufficient-evidence refusal. Close by distinguishing implemented facts from the roadmap: version metadata/filtering, document lifecycle, hybrid/learned reranking only after measurements, security, concurrency and production observability.

23. Final Know-My-Project Cheat Sheet

- **Definition:** local, cited, grounded RAG prototype for document support.
- **Actual corpus:** four academic/sample PDFs; 9,351 chunks/vectors; not product documentation.
- **Ingestion:** Streamlit bytes → SHA-256 save → parser → `ChunkRecord` → MiniLM → FAISS + pickle.

- **Raw source:** data/uploads; **chunks/metadata:** data/faiss/index.pkl; **vectors:** data/faiss/index.faiss; **manifest:** data/metadata/uploaded_files.json.
- **Chunking:** structure-aware, 1,000 characters, up to 200-character non-table semantic overlap; id file:page:row:chunk.
- **Embeddings:** local all-MiniLM-L6-v2, batch 32, normalized 384-d float32.
- **FAISS:** exact IndexFlatIP; position maps to pickle record. No PDF/text inside FAISS.
- **Retrieval:** query rewrite → 5× candidate pool → semantic + lexical heuristics → penalties/dedupe/adjacency → top-K → >=0.35 gate.
- **LLM:** active local Ollama qwen3:8b; Gemini optional path. Prompt grounding and pre-generation gates both matter.
- **Memory:** Streamlit session only, last six filtered messages; never factual evidence.
- **LangGraph:** deterministic branch orchestration, not autonomous agency.
- **Citations:** Python-built from filename/page/row/chunk/id retrieval metadata.
- **Chapter issue:** structural chunks can semantically rank; penalties, dedupe and adjacent substantive expansion mitigate it without weakening refusal.
- **Most important limits:** no individual update/delete, no atomic ingest, no version/ACL/tenant model, no OCR/hybrid/learned reranker, no production observability, no real-product evaluation.